CA-3 · Foundation of Agentic AI

Reading the Rig

Agentic AI for Daily Drilling Reports: finding non-productive time with a LangGraph multi-agent system

Project report · Proof of concept

Rahul Narkhede (24070126146)
Ranjan Verma (24070126149)

TY B.Tech AI-ML, Division B
Symbiosis Institute of Technology, Pune
October 2026

Repository: github.com/ranjanvrma/rig-sight · Live demo: rig-sight.streamlit.app

All data in this project is synthetic. This is a classroom proof of concept, not a certified engineering or safety system.

Contents

Contents 2
Abstract 3
1. Introduction and domain background 3
1.1 Why oil and gas drilling 3
1.2 The Daily Drilling Report 3
1.3 Non-productive time 3
2. Problem statement 4
3. Existing process and identified inefficiencies 4
4. Proposed solution and objectives 4
5. Requirements and technology stack 5
5.1 Functional requirements 5
5.2 Technology stack 5
6. System architecture and data flow 5
7. Agent roles and orchestration 8
8. Data model, synthetic dataset and reference files 8
8.1 Shared state (DDRState) 8
8.2 Synthetic dataset 9
9. Implementation details 9
10. Deterministic tools and model usage 10
11. Critic validation, retries and human-in-the-loop review 10
12. Streamlit interface 11
12.1 Deployment 11
12.2 Screens 11
13. Testing methodology and test cases 16
14. Evaluation results 17
Interpretation 17
15. Limitations and responsible use 18
16. Future enhancements 18
17. Conclusion 18
References 19
Appendix A. How to run 19

Abstract

Every drilling rig sends a Daily Drilling Report (DDR) each morning, and most of it is free text typed by the crew. Hours that are paid for but do not move the well forward, called non-productive time (NPT), hide inside those sentences, so they are rarely totalled and lessons are lost between wells. This project builds a working proof of concept in which a team of LangGraph agents reads a DDR, plans the work, looks up reference data and past incidents, computes every number with deterministic Python tools, checks its own output with a Critic that can send work back with specific feedback, and pauses for a drilling engineer to approve or reject an evidence-backed insight report. A local language model (qwen2.5:3b through Ollama) is used only to label log lines; a keyword classifier is the fallback and is always reported as such.

On 15 synthetic reports with a known answer key, the full graph had a mean NPT error of 0.47 h per report, got every planted NPT line's category right (100%), caught both deliberately planted log errors (2/2) and produced time logs that add to 24 h in 15/15 reports. A single-prompt chatbot using the same model had a mean error of 3.83 h, and its time log added to 24 h in only 1/15 reports. The remaining errors come from the model confidently labelling planned work as NPT; the engineer review step corrects them. 48 automated tests cover the tools, the Critic, retries, human review, malformed input and the user interface.

1. Introduction and domain background

1.1 Why oil and gas drilling

Drilling and completing wells takes 40 to 50 percent of an offshore operator's capital spend (Source 9), and a drillship cost close to $400,000 a day in late 2025 (Source 10), roughly $16,000 for every hour the well does not progress. Operators that tracked lost time closely cut NPT from about 16 percent to 9 percent within a year (Source 9). The industry has also started buying agentic AI: in 2025 ADNOC awarded AIQ a $340 million contract to deploy agentic systems across more than 28 fields (Source 11), while the SPE ATCE 2026 programme openly asks how far agents can be trusted with operational decisions (Source 4). That combination, real money and an open question of trust, makes the domain a good fit for an agentic proof of concept with a human in the loop.

1.2 The Daily Drilling Report

A DDR covers the previous 24 hours: an hour-by-hour time log, depth drilled, bit and mud data, weather, crew on board and the cost of the day. The IADC has been reworking the format so that activities are tracked in more detail (Source 2). Most of the time log is free text, for example:

```
06:00-08:30  Drilled 12-1/4 in hole from 2,450 m to 2,610 m
08:30-14:00  POOH, MWD tool stopped sending data
14:00-16:00  Waited on weather, wind above crane limit
```

The second and third lines are both lost time, but nothing in the report adds them up.

1.3 Non-productive time

NPT is rig time that is paid for while the well does not move forward. In a study of wells on the Lobo trend, stuck pipe and lost circulation alone made up about 74 percent of NPT (Source 1); other common

causes are tool failure, waiting on weather, waiting on equipment and well control (Source 3). This project uses those six categories.

2. Problem statement

How can a drilling engineer get a reliable, evidence-backed account of lost time from every daily drilling report without reading each one line by line?

Reports arrive every day from many rigs as free text. The account must give exact hours per NPT category, show which report line each claim comes from, compare progress and cost with plan, suggest a likely cause from past events, and leave the final decision to the engineer.

3. Existing process and identified inefficiencies

- **Manual reading.** An engineer reads each report by eye; with many rigs this does not scale.
- **Small losses are not totalled.** Half an hour here and there is never added up across a day or a well.
- **Inconsistent labels.** Different people classify the same event differently, so trends are unreliable.
- **No memory.** Causes and fixes from past wells are not reused, so the same failure returns on the next well.
- **Data errors pass unnoticed.** Gaps, overlaps and typos in the time log are not checked systematically.
- **A plain chatbot does not fix this.** One prompt and one answer: the model has to read and count, cannot check itself and gives no proof. Section 13 measures this.

4. Proposed solution and objectives

A LangGraph multi-agent system in which each agent has one job and all of them share one state. The guiding rule is "the model reads, Python counts": the language model only interprets free text, and every number is computed by a tool so the Critic can check it exactly.

Objectives

- Ingest a DDR, parse its time log and plan the processing tasks.
- Look up the well plan, NPT categories, daily budget and similar past incidents.
- Classify every activity as productive or one of six NPT categories, with a confidence score.
- Compute total hours, NPT by category, rate of penetration (ROP), depth against plan and cost against budget with deterministic tools.
- Validate the result with a Critic, retry with specific feedback when the problem is fixable, and report unfixable source-data errors instead of repairing them silently.
- Draft an insight report that separates observed data, inferred causes, validation results and suggested actions, with a source line for every NPT claim.
- Pause for the engineer to approve, or reject with a comment that the system acts on.
- Measure accuracy against a known answer key and compare with a single-prompt chatbot on the same model.

5. Requirements and technology stack

5.1 Functional requirements

Table 1. Functional requirements and the code that meets them

ID	Requirement	Where it is met
F1	Ingest a synthetic DDR (file or pasted text)	ingest_report, app custom report
F2	Parse time-log entries; report malformed lines and invalid timestamps	parse_time_log
F3	Plan the processing tasks	planner
F4	Research well plan, NPT codes, budget, past incidents	research + lookup tools
F5	Classify productive vs NPT activities	classify_activities (LLM or mock)
F6	Deterministic totals, NPT, ROP, depth variance, cost	sum_hours, calc_rop, depth_vs_plan, cost_vs_budget
F7	Real orchestration	route() over DDRState, conditional edges
F8	Critic validation	critic
F9	Retry with feedback, limited	fix_hints, MAX_RETRIES = 3
F10	Evidence-backed insight report	report_writer
F11	Human approve / reject with comments	human_review, interrupt()
F12	UI with agent activity, results, charts, evidence, review	app.py
F13	Clean, NPT and planted-error test cases	data/, tests/
F14	Honest fallback when no LLM is available	llm.py mock, 'labelled by' in every report

5.2 Technology stack

Table 2. Technology stack

Layer	Choice	Why
Language	Python (tested on 3.14 and 3.11)	Course requirement; rich tooling
Orchestration	LangGraph 1.2 (StateGraph, ToolNode, interrupt, checkpointer)	Explicit state graph with loops, tools and human pause
Messages / tools	LangChain Core 1.6	Tool schemas and tool messages for the ToolNode
Language model	Ollama 0.40 with qwen2.5:3b (local, free)	No paid API; data stays on the laptop; fits a 4 GB GPU
Fallback	Keyword classifier built from npt_codes.json	Demo keeps working without a model
Storage	SQLite (incident memory, LangGraph checkpoints)	Built into Python
UI	Streamlit 1.55+, Altair charts	Course requirement
Testing	pytest (48 tests), Streamlit AppTest	Repeatable checks

Hardware used: a laptop with an NVIDIA RTX 3050 (4 GB). The 8B model suggested in the original design does not fit in 4 GB, so a 3B model was chosen.

6. System architecture and data flow

The design is hub and spoke (Figure 1). The Orchestrator sits in the middle, reads the shared state and decides which agent works next. Every agent hands back to the Orchestrator; agents never call each other. Reference data, incident memory and the checkpointer live outside the graph. The Streamlit UI starts a run, streams every node's update and resumes the graph when the engineer decides.

A normal run, step by step:

1. Streamlit calls app.stream(initial_state, thread_id). ingest_report reads the file and parses the header.
2. The Orchestrator sends the state to the Planner, which writes the task list.
3. Research looks up the well plan, budget, NPT codes and similar incidents. Missing data becomes a warning.
4. The Executor asks the ToolNode to parse the time log, then to classify activities and run the four maths tools in parallel, then totals NPT by category in Python.
5. The Critic checks the result. A fixable FAIL goes back to the Executor with structured fix hints (at most three times); a PASS, an unfixable FAIL or used-up retries go to report_writer.
6. report_writer drafts the Daily Insight Report. human_review calls interrupt() and the graph stops.
7. The engineer approves (END) or rejects with a comment; a rejection clears the work and the Executor re-runs with the engineer's line overrides.

Figure 1, on the next page, shows the full architecture and the numbered data flow.

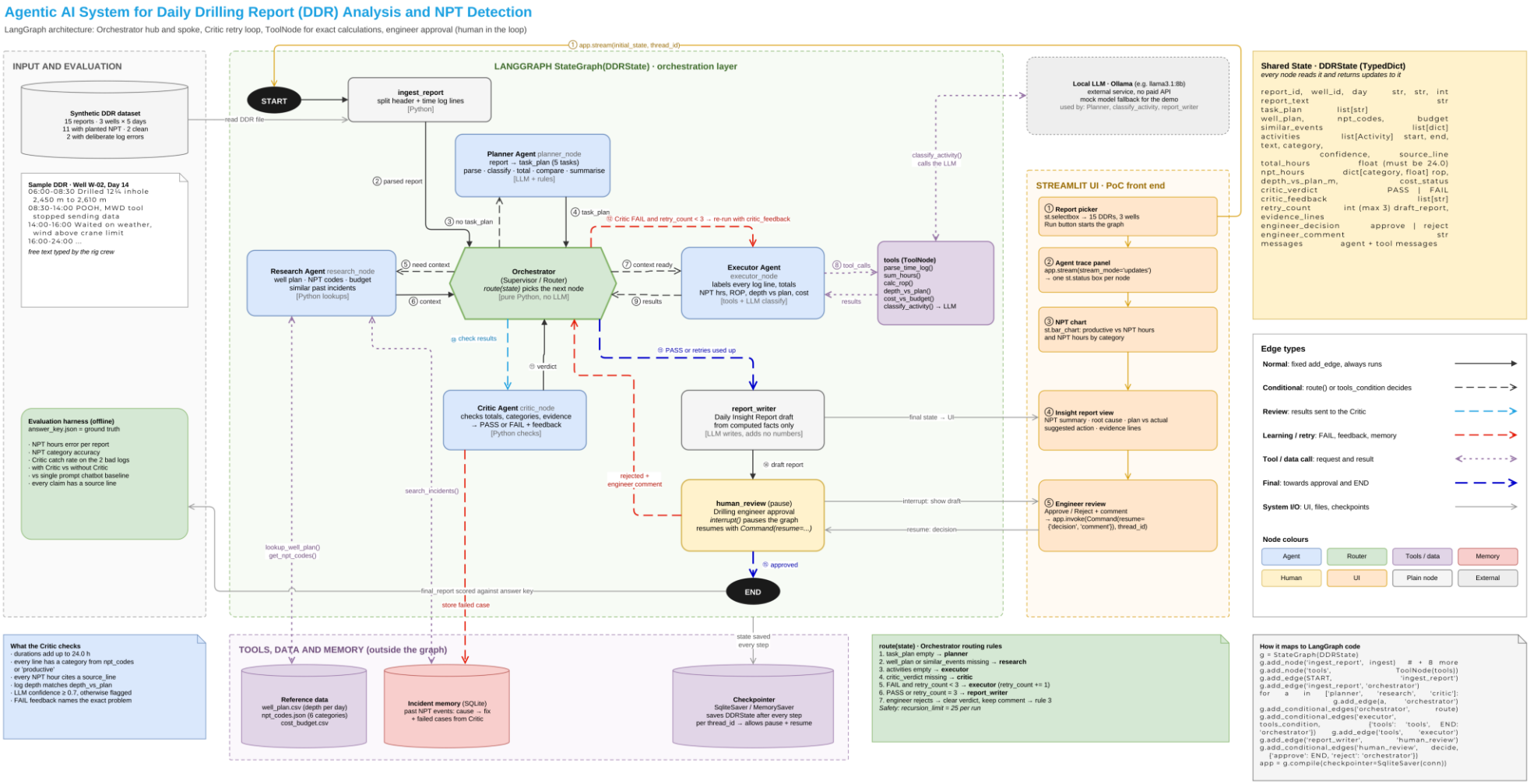


Figure 1. Architecture of the system (the team's design diagram)

The diagram is the team's original design. The final code follows it, with these differences: the Planner is rule based rather than LLM based; recursion_limit is 50, not 25 (the worst case needs 32 steps); the local model is qwen2.5:3b, because llama3.1:8b does not fit in 4 GB of GPU memory; classification runs as one classify_activities tool over all lines; the chart is an Altair timeline plus NPT by category; and the Critic also checks gaps, overlaps, unparseable lines and the header (Section 11).

7. Agent roles and orchestration

Table 3. Agent roles

Role	Node	Responsibility	Uses the LLM?
Orchestrator	orchestrator + route()	Reads DDRState and picks the next node	No (plain Python)
Planner	planner	Task list that adapts to the report: REMARKS present, drilling lines, depth, cost	No (rules)
Research	research	Well plan, budget, NPT codes, similar past incidents	No
Executor	executor + tools	Drives the six tools over three visits and totals NPT	Through classify only
Critic	critic	Hard checks, specific feedback, decides whether a retry can help, logs failed cases	No
Report writer	report_writer	Insight report from computed facts	Optional rephrase, off by default
Engineer	human_review	Approve, or reject with a comment	Human

Routing rules in route(), checked top to bottom (the first match wins):

```
1. no task_plan                                  -> planner
2. no well_plan or similar_events                -> research
3. activities is None                            -> executor
4. no critic_verdict                             -> critic
5. FAIL, retryable and retry_count < 3           -> executor (with fix_hints)
6. otherwise (PASS, unfixable FAIL, retries used) -> report_writer -> human_review
7. engineer rejects -> activities and verdict cleared, overrides kept -> rule 3
```

The Orchestrator is code rather than a model because routing only needs simple checks on the state; code is predictable, free, testable and easy to explain. recursion_limit is 50. Measured step counts are 16 for a normal report, 24 with one retry and 32 for the worst case (the Critic failing every time), which still reaches the engineer.

8. Data model, synthetic dataset and reference files

8.1 Shared state (DDRState)

Table 4. Main fields of the shared state

Field(s)	Type	Meaning
report_id, well_id, day, report_text, header	str, int, dict	Which report and its parsed header
task_plan	list[str]	Planner output
well_plan, budget, npt_codes, similar_events	dict, list	Research output
reference_warnings	list[str]	Missing or unreadable reference data
activities	list[dict] | None	One item per log line: start, end, hours, text, category, confidence, source_line, labelled_by, note
parse_issues	list[dict]	Malformed lines and invalid timestamps
total_hours, npt_hours, total_npt, rop, depth_vs_plan_m, cost_status	float, dict	Tool results
critic_verdict, critic_feedback, critic_warnings, critic_retryable	str, list, bool	Critic output
fix_hints, retry_count, attempts	list, int, list	Retry loop state and history
draft_report, evidence_lines	str, list[int]	Report writer output

overrides, engineer_decision, engineer_comment	dict, str	Human review
trace, messages	list	UI trace and agent / tool messages for the ToolNode

8.2 Synthetic dataset

data/generate_data.py writes every file. Each line is written together with its true label, so the answer key does not depend on the agents' parser or classifier. Real DDRs are confidential, which is a second reason to use synthetic data.

Table 5. The 15 reports (3 wells × 5 days; 2 clean)

Type	Reports	Notes
Clean days	W-01 D10, W-03 D11	No NPT
Planted NPT	11 reports	All six categories: tool failure, stuck pipe, lost circulation, waiting on weather, waiting on equipment, well control
Planted error: hidden line	W-02 D14	20:30-21:00 entry only in REMARKS, so the log adds to 23.5 h
Planted error: overlap	W-03 D13	Two lines overlap by 30 min, so the log adds to 24.5 h
Trap	W-03 D14	'Waited on cement (WOC)' is planned work, not NPT

Reference files: well_plan.csv (planned end-of-day depth), cost_budget.csv (daily budget per well), npt_codes.json (six categories with crew keywords), incidents.sqlite (nine past incidents with cause and fix, plus empty tables for Critic failed cases and engineer decisions) and answer_key.json (true label per line, NPT hours, depth against plan and ROP). The app works on a copy of the memory database in runtime/, so using the demo never changes the committed data.

```
WELL W-02  DAY 14  DEPTH 2,610 m  COST USD 418,000
00:00-06:00 Drilled 12-1/4 in hole from 2,300 m to 2,450 m
06:00-08:30 Drilled 12-1/4 in hole from 2,450 m to 2,610 m
08:30-14:00 POOH, MWD tool stopped sending data
14:00-16:00 Waited on weather, wind above crane limit
16:00-20:30 Tested BOP as per schedule
21:00-24:00 Slipped and cut drill line, serviced rig
REMARKS: 20:30-21:00 Waited on tong repair before slip and cut
```

Report W-02 Day 14 (above). The true answer is 24.0 h logged and 8.0 h NPT: tool failure 5.5 h, waiting on weather 2.0 h, waiting on equipment 0.5 h; 40 m behind plan; ROP 36.5 m/h.

9. Implementation details

Table 6. Repository modules

File	Contents
graph.py	DDRState, the eight nodes, route(), graph build, run helper
tools.py	Executor tools (ToolNode) and Research lookups; all arithmetic
llm.py	The only place a model is called: Ollama client, mock classifier, summary, chatbot baseline
memory.py	SQLite incident memory, failed cases, engineer feedback, runtime copy
app.py	Streamlit UI
evaluate.py	Scores the 15 reports in three configurations
data/generate_data.py	Writes the dataset, answer key and reference files
tests/	48 pytest tests

The Executor shows how tool use works with LangGraph. On its first visit it writes an AIMessage with a parse_time_log tool call; tools_condition sends it to the ToolNode, which returns a ToolMessage. On the second visit it issues five tool calls at once (classify_activities, sum_hours, calc_rop, depth_vs_plan, cost_vs_budget). On the third visit it reads the results, totals NPT by category in Python and writes a plain message, so tools_condition hands control back to the Orchestrator.

Engineer comments are turned into line overrides: a comment such as "line 7 is planned, not NPT" becomes {"7": "productive"}, and the re-run labels that line as set by the engineer (confidence 1.0, labelled by 'engineer').

10. Deterministic tools and model usage

Table 7. Tools

Tool	What it does
parse_time_log	Splits HH:MM-HH:MM lines into activities with source line numbers. Malformed lines and impossible or reversed times are returned as issues, never guessed at. REMARKS are read only to fill a gap the Critic named; an overlap the Critic named is trimmed. Both changes are noted on the activity and shown in the report.
classify_activities	Labels each activity (LLM or mock) with a category and confidence; engineer overrides win
sum_hours	Total logged hours (must be 24.0)
calc_rop	Metres drilled ÷ drilling hours from 'Drilled ... from X m to Y m' lines
depth_vs_plan	End depth minus planned depth; null when either is unknown
cost_vs_budget	Day cost against the daily budget
lookup_well_plan, get_npt_codes, search_incidents	Research lookups; missing files or rows return null and a warning

The language model is used in one place by default, with a second place available but switched off:

- **classify_activities:** reads one log line and returns a category and a confidence score as JSON. Invalid output falls back to the mock for that line, and the line records which one was used.
- **report_writer (off by default, RIG_LLM_SUMMARY=1):** may rephrase the computed facts into a short summary. A guard discards any summary containing a number that is not in the computed facts. During testing the model kept every number but attached them to the wrong categories (it wrote that waiting on weather and waiting on equipment took "3.5 hours, 2 hours respectively" when the facts were 2.0 h and 3.5 h). A number guard cannot detect that, so the template summary is the default. The report states which was used.

The model never does arithmetic. When Ollama is not running, the app uses the mock and says so in the sidebar, in every activity ('labelled_by') and in the report ('Lines labelled by: mock').

11. Critic validation, retries and human-in-the-loop review

Table 8. What the Critic checks

Check	On failure
Header present and valid	FAIL, not retryable (source error)
Every time-log line parseable, times valid	FAIL, not retryable; line marked in red in the UI
At least one valid entry	FAIL
Logged hours add to 24.0	FAIL
No gaps between activities	FAIL + gap hint (Executor reads REMARKS for that window)

No overlaps	FAIL + overlap hint (Executor trims the later line, noted for the crew)
Every activity has a valid category	FAIL
Every NPT claim's text is on its cited source line	FAIL
NPT total equals the sum of the NPT lines	FAIL
Deepest drilled depth equals the header depth	FAIL
Model confidence ≥ 0.7	Warning shown to the engineer (not a FAIL)

A FAIL is retried only when the Critic has a new hint for the Executor to try; otherwise the draft goes to the engineer with the unresolved problems listed. This avoids three identical retries on an error only the rig crew can fix, and it keeps source errors visible instead of hiding them. Every FAIL is also written to the failed_cases table in memory.

Human review uses LangGraph's interrupt(): the graph stops inside human_review with its state saved by the checkpointer, and the UI resumes it with Command(resume={decision, comment}) on the same thread_id. Approve ends the run. Reject clears the activities and the verdict, keeps the comment and its overrides, and sends the work back through the Executor and the Critic before the engineer sees it again. Every decision is stored.

12. Streamlit interface

The interface is a single Streamlit page with four tabs: Analyse report, Evaluation, Memory and Architecture.

12.1 Deployment

The application is also deployed on Streamlit Community Cloud at https://rig-sight.streamlit.app/, built from the main branch of the GitHub repository with app.py as the entry point; a push to main redeploys it. It was checked on 19 October 2026: W-02 Day 14 gave 8.0 h NPT, the Critic failed the 23.5 h first pass and the retry recovered the missing half hour.

- **Model:** the cloud has no Ollama, so the hosted app runs on the mock keyword classifier. The sidebar and every report say so, and choosing Ollama shows a warning and falls back to the mock.
- **Results:** the Evaluation tab shows the committed qwen2.5:3b and mock results; re-running the evaluation there scores the mock only.
- **State:** memory and checkpoints are written to temporary server storage. They are lost when the app restarts, sleeps or redeploys, and are shared by everyone using it at the same time.
- **Use:** the hosted app shows the workflow with no setup; the results with the real model, including the engineer correcting the model's mistake, come from running it locally with Ollama.

12.2 Screens

All screenshots below were taken from the application running locally with the qwen2.5:3b model.

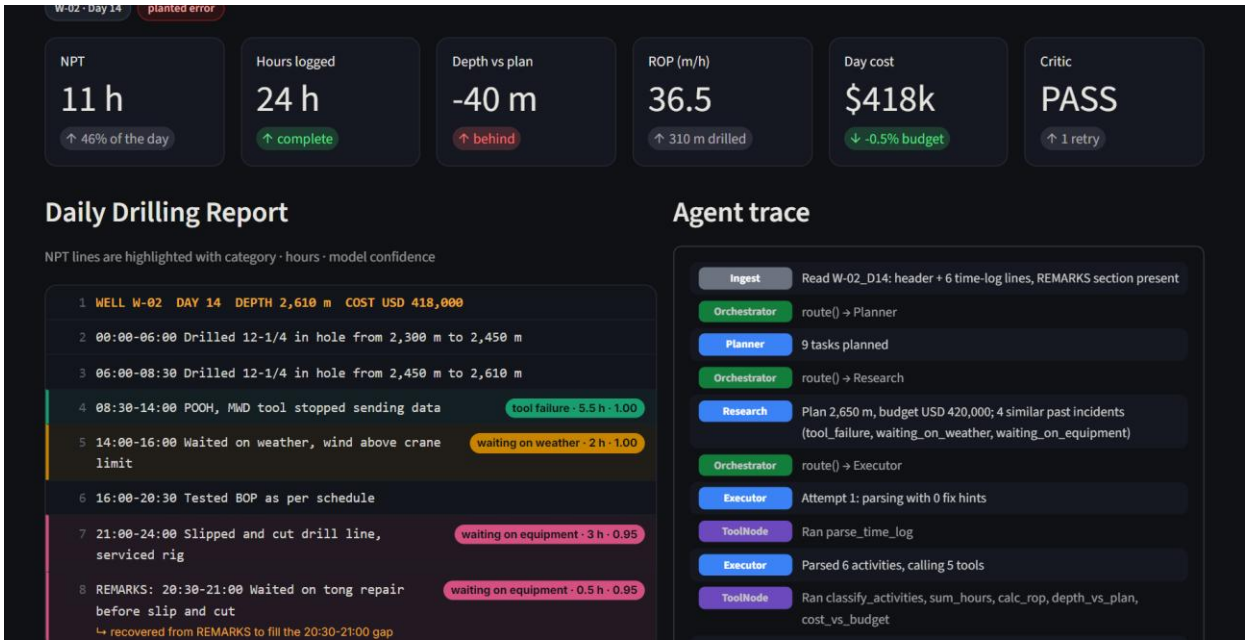


Figure 2. W-02 Day 14 after a run: KPI tiles, the time log with NPT lines labelled (category · hours · confidence) and the line recovered from REMARKS, and the live agent trace including the Critic's FAIL and the retry

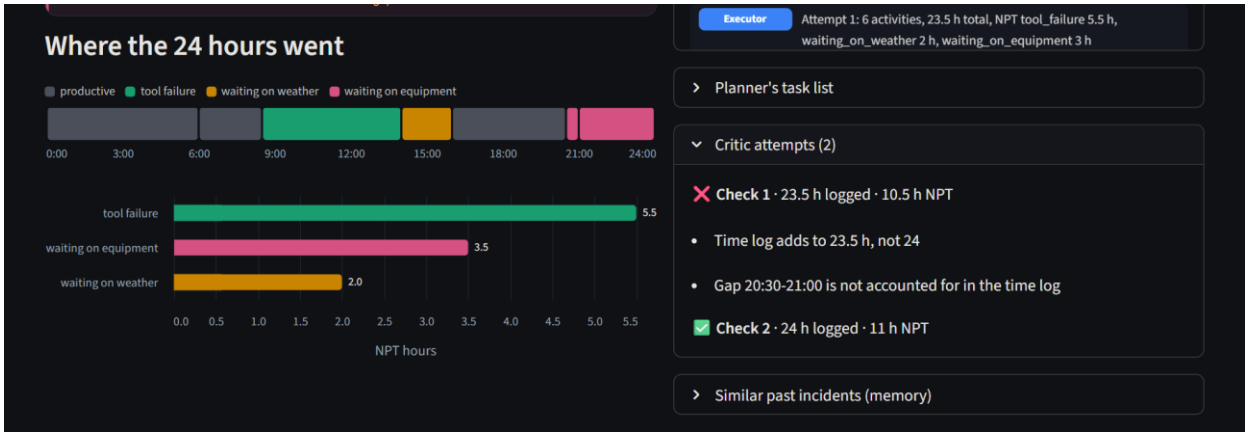


Figure 3. Where the 24 hours went: timeline and NPT by category, the Planner's tasks, the Critic's two checks and similar past incidents from memory

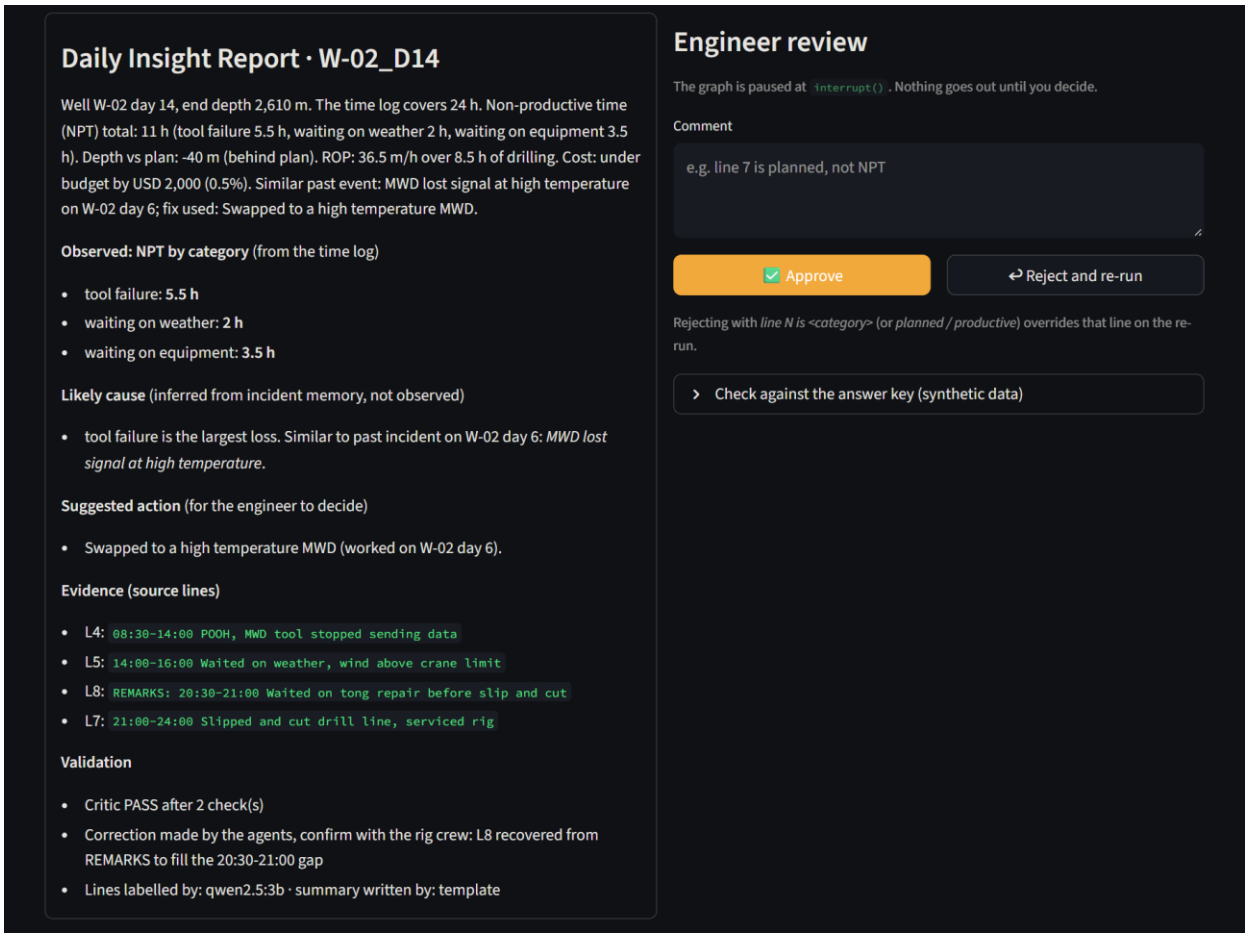


Figure 4. First draft of the Daily Insight Report and the engineer review panel. The model has labelled line 7 (slip and cut) as NPT



Figure 5. After the engineer rejected with 'line 7 is planned, not NPT' and then approved: 8.0 h NPT, matching the answer key

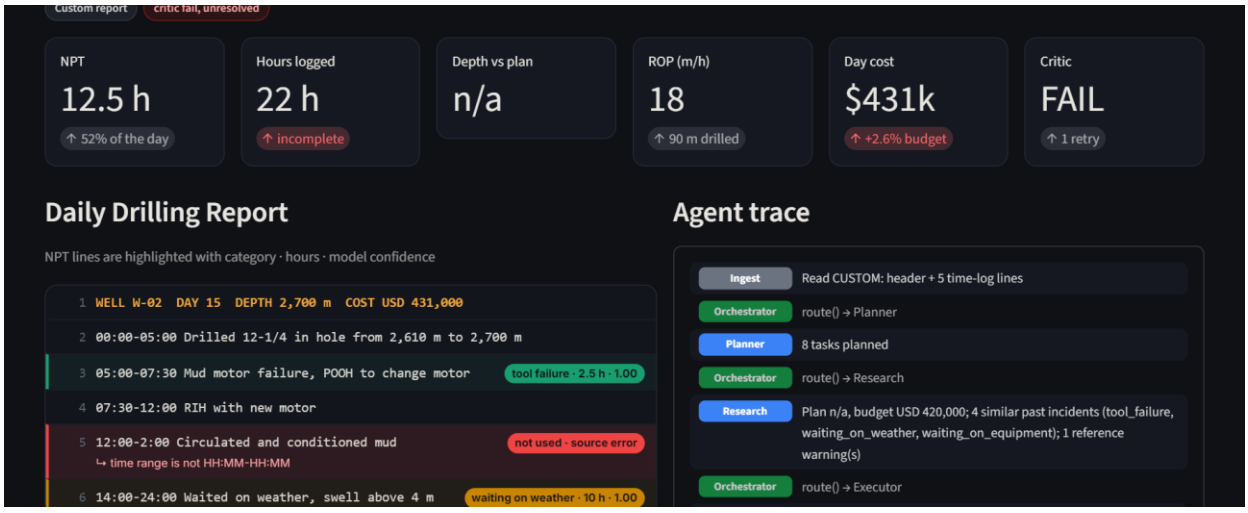


Figure 6. A pasted report with an invalid time range (12:00-2:00). The line is marked as a source error and left unrepaired; the Critic fails without a pointless retry

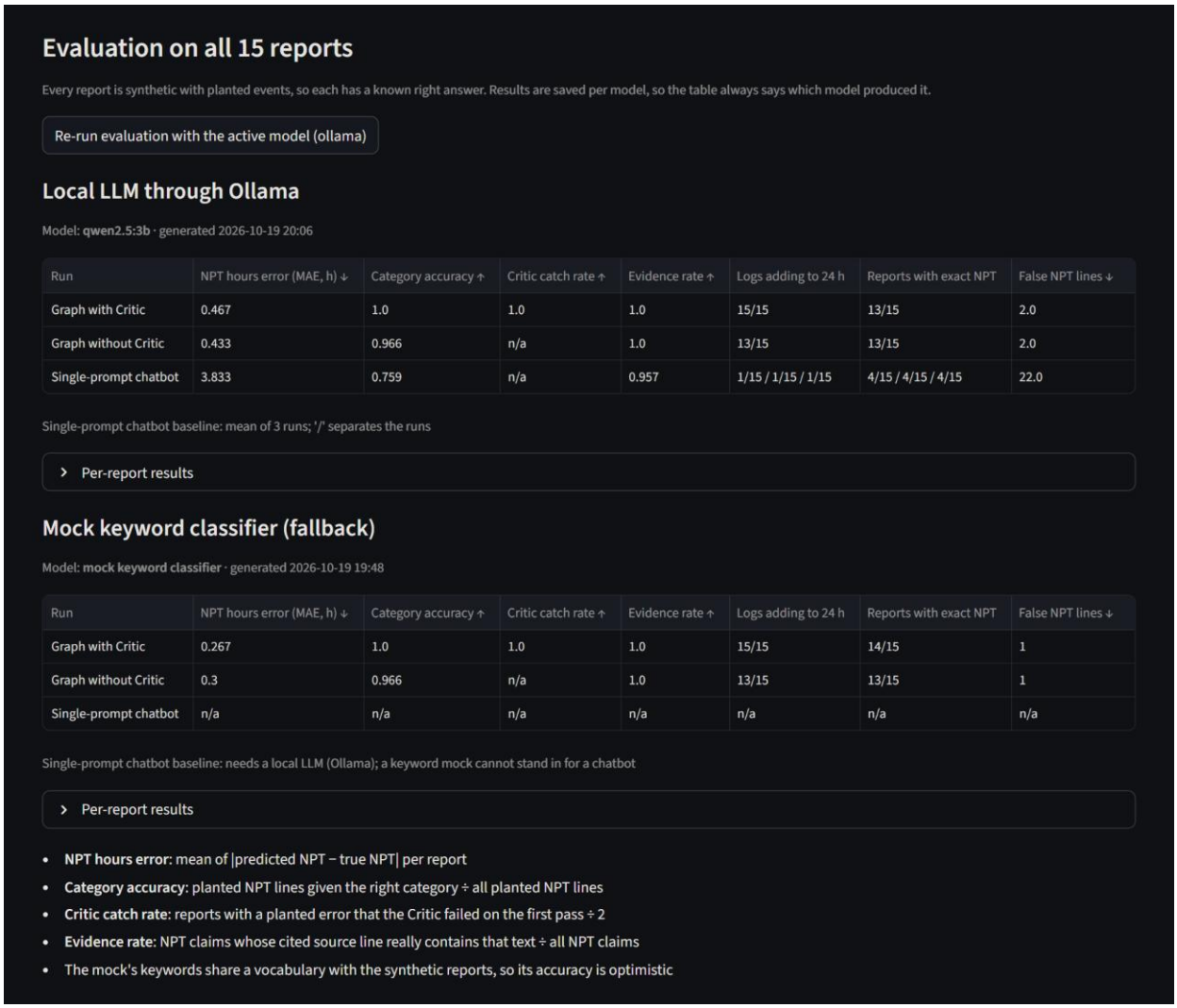
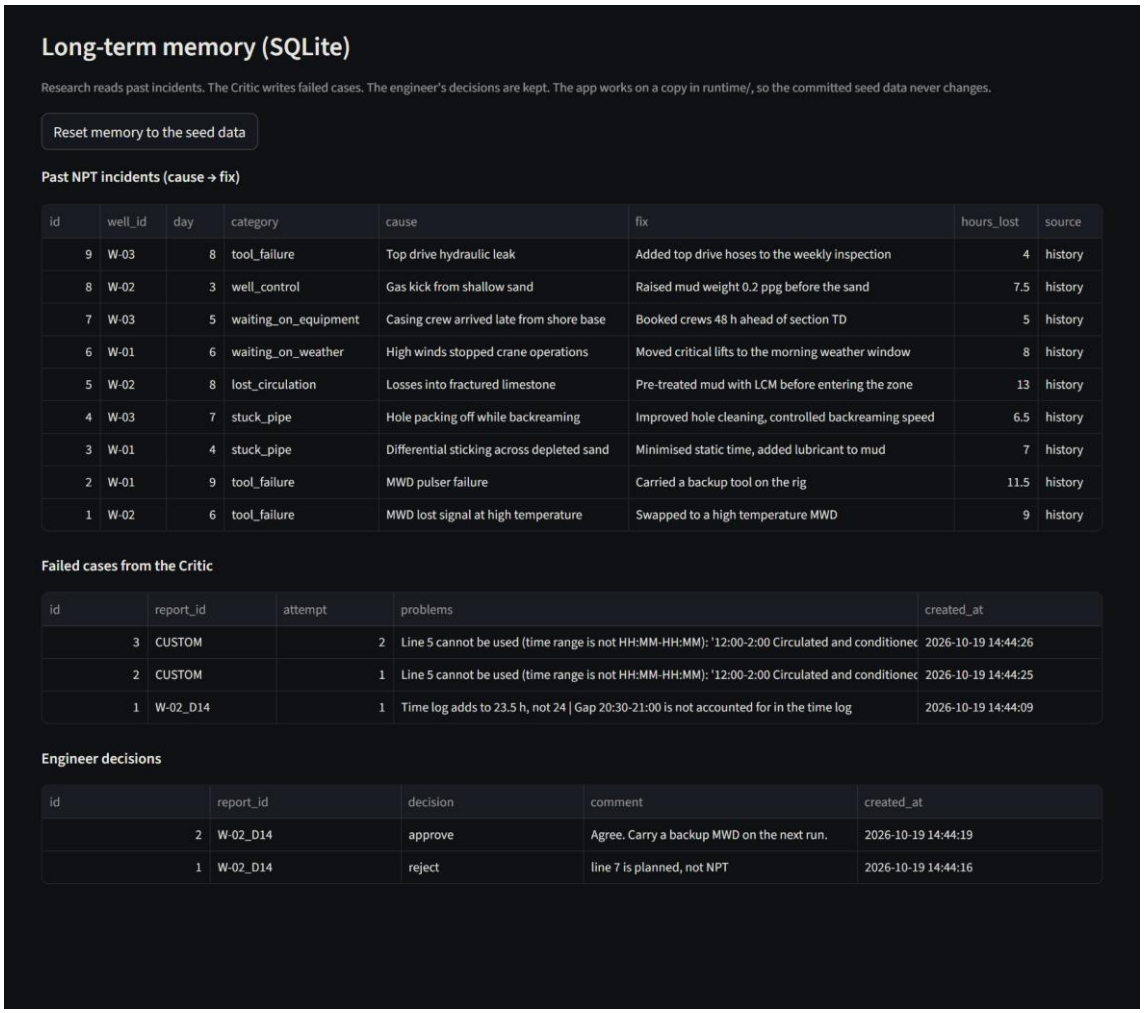


Figure 7. Evaluation tab with the saved results for the local model and the mock



Long-term memory (SQLite)

Research reads past incidents. The Critic writes failed cases. The engineer's decisions are kept. The app works on a copy in runtime/, so the committed seed data never changes.

Reset memory to the seed data

Past NPT incidents (cause → fix)

id	well_id	day	category	cause	fix	hours_lost	source
9	W-03	8	tool_failure	Top drive hydraulic leak	Added top drive hoses to the weekly inspection	4	history
8	W-02	3	well_control	Gas kick from shallow sand	Raised mud weight 0.2 ppg before the sand	7.5	history
7	W-03	5	waiting_on_equipment	Casing crew arrived late from shore base	Booked crews 48 h ahead of section TD	5	history
6	W-01	6	waiting_on_weather	High winds stopped crane operations	Moved critical lifts to the morning weather window	8	history
5	W-02	8	lost_circulation	Losses into fractured limestone	Pre-treated mud with LCM before entering the zone	13	history
4	W-03	7	stuck_pipe	Hole packing off while backreaming	Improved hole cleaning, controlled backreaming speed	6.5	history
3	W-01	4	stuck_pipe	Differential sticking across depleted sand	Minimised static time, added lubricant to mud	7	history
2	W-01	9	tool_failure	MWD pulser failure	Carried a backup tool on the rig	11.5	history
1	W-02	6	tool_failure	MWD lost signal at high temperature	Swapped to a high temperature MWD	9	history

Failed cases from the Critic

id	report_id	attempt	problems	created_at
3	CUSTOM	2	Line 5 cannot be used (time range is not HH:MM-HH:MM): '12:00-2:00 Circulated and conditionec	2026-10-19 14:44:26
2	CUSTOM	1	Line 5 cannot be used (time range is not HH:MM-HH:MM): '12:00-2:00 Circulated and conditionec	2026-10-19 14:44:25
1	W-02_D14	1	Time log adds to 23.5 h, not 24 | Gap 20:30-21:00 is not accounted for in the time log	2026-10-19 14:44:09

Engineer decisions

id	report_id	decision	comment	created_at
2	W-02_D14	approve	Agree. Carry a backup MWD on the next run.	2026-10-19 14:44:19
1	W-02_D14	reject	line 7 is planned, not NPT	2026-10-19 14:44:16

Figure 8. Memory tab: seed incidents, failed cases written by the Critic and the engineer's decisions from the runs above

13. Testing methodology and test cases

Automated tests are in tests/ and run with python -m pytest. All 48 pass. They use the mock classifier and a temporary memory database, so they are deterministic and do not touch the committed data or a running model.

Table 9. Automated tests

File	What is tested
test_tools.py	Header parsing; time-log parsing; malformed lines, wrong format, impossible and reversed times reported as issues; REMARKS read only with a gap hint; overlap trimming with a note; sums, ROP, depth and cost; missing, invalid and unknown reference data
test_graph.py	All 15 reports: 24 h, Critic PASS, depth and ROP equal to the answer key, every NPT claim on its own line, NPT total equals the NPT lines. Mock exact on 14/15 (the WOC trap flagged as low confidence). Clean day. Critic catches the hidden line and the overlap and the retry fixes them, with the correction disclosed. Failed cases stored. No-Critic ablation. Approve ends the graph. Reject with an override re-runs and corrects. Invalid timestamp, missing header, empty log and unknown well handled. Worst case stops after 3 retries and still reaches the engineer. Mock reported as mock. LLM summary with an invented number rejected; one with only computed numbers kept. Ollama unavailable falls back to the mock
test_dataset.py	Dataset shape (15 reports, 2 clean, 2 planted errors, all six categories), answer key consistency, generator reproducible byte for byte
test_app.py	Streamlit app loads, runs W-02 D14 to the engineer pause and approves; custom report shows the unresolved source error; other tabs render

Evaluation (evaluate.py) runs the same 15 reports through three configurations with the same model: the full graph, the graph without the Critic (route goes straight from the Executor to the report writer), and a single-prompt chatbot that is asked for NPT hours by category, the total hours and the line numbers it used. The chatbot prompt contains no example numbers, so it is not nudged towards an answer.

14. Evaluation results

Metrics:

- **NPT hours error:** mean of |predicted NPT − true NPT| per report, in hours.
- **Category accuracy:** planted NPT lines given the right category ÷ all planted NPT lines. The chatbot does not give a category per line, so a cited line counts as correct if its true category is among those it reported (in its favour).
- **Critic catch rate:** planted-error reports the Critic failed on the first pass ÷ 2.
- **Evidence rate:** NPT claims whose cited source line really contains the claimed text ÷ all NPT claims.
- **Logs adding to 24 h; reports with exact NPT; false NPT lines** (productive lines claimed as NPT).

Table 10. Results on the 15 reports with the local model (2026-10-19 20:06)

Run (qwen2.5:3b)	NPT error	Category acc.	Critic catch	Evidence	24 h logs	Exact NPT	False NPT
Graph with Critic	0.47 h	100%	2 / 2	100%	15/15	13/15	2
Graph without Critic	0.43 h	96.6%	n/a	100%	13/15	13/15	2
Single-prompt chatbot	3.83 h	75.9%	n/a	95.7%	1/15	4/15	22

Table 11. Results with the mock fallback (no chatbot baseline: a keyword mock cannot stand in for a chatbot)

Run (mock keyword classifier)	NPT error	Category acc.	Critic catch	Evidence	24 h logs	Exact NPT
Graph with Critic	0.27 h	100%	2 / 2	100%	15/15	14/15
Graph without Critic	0.30 h	96.6%	n/a	100%	13/15	13/15

Interpretation

- **The agent graph is about eight times more accurate than a chatbot on the same model** (0.47 h against 3.83 h). The chatbot's log added to 24 h in only 1/15 reports: for complete 24-hour logs it reported totals between 11 h and 126 h, which is why arithmetic is kept in Python.
- **The Critic caught both planted errors (2/2)** and the retry fixed them. Without it, W-02 D14 reports 23.5 h and W-03 D13 reports 24.5 h, so only 13 of 15 logs add to 24 h.
- **Why 'without Critic' has a slightly lower MAE:** 0.43 h against 0.47 h, because two mistakes cancel on W-02 D14. Without the Critic the missing 0.5 h offsets part of a separate 3 h mislabel; with the Critic the 0.5 h is correctly recovered and only the mislabel remains. The no-Critic run is wrong in two ways, which category accuracy and the 24 h check show.
- **The two remaining errors are confident mislabels of planned work:** 'slipped and cut drill line, serviced rig' (W-02 D14, 3 h) and 'waited on cement' (W-03 D14, 4 h), both labelled waiting on equipment with confidence of 0.9 or more, so the 0.7 confidence floor does not flag them. The engineer fixes each with one reject comment (Figure 5). This is the main argument for keeping a human in the loop.

- **Repeatability:** the graph's results were identical across three separate runs on different occasions (temperature 0). The chatbot's three repeats within one evaluation run were identical too, but separate runs on different occasions gave mean errors of 4.43 h, 3.77 h and 3.83 h; the table shows the latest saved run.
- **LLM summaries misattribute numbers:** see Section 10. This was found from a real screenshot of the app, and is why the summary is a template by default.
- **Tried and rejected:** a 'decide planned or unplanned first' classification prompt scored 60 of 73 lines against 71 of 73 for the simple prompt, so the simple prompt was kept rather than tuning the prompt to the test set.

15. Limitations and responsible use

- Synthetic data only: 15 reports, three wells, one report format and six NPT categories. Results on real, messier reports would be lower.
- The 3B local model mislabels some planned work as NPT with high confidence, so confidence alone is not a safe filter. The mock classifier only knows its keywords, and its accuracy is optimistic because the keywords and the synthetic reports share a vocabulary.
- The parser expects HH:MM-HH:MM entries. Other formats are reported as errors rather than understood.
- Overlap trimming is a guess about which line is wrong; it is disclosed in the report for the crew to confirm.
- Engineer overrides apply to that run; they are stored but not yet learned from.
- The likely cause is inferred from similar past incidents, not observed, and is labelled as such.
- This is not a certified engineering or safety tool. Every report needs an engineer's approval, and the system never sends anything on its own.

16. Future enhancements

- Live rig data feeds (for example WITSML) and many wells at once.
- Learning from engineer corrections, for example as few-shot examples drawn from the engineer_feedback table.
- A larger or fine-tuned model for classification, evaluated on a held-out set.
- A morning NPT alert to the drilling team and trends across wells: recurring causes and cost per category.
- MCP connectors so the agents can read company systems (the DDR database, maintenance logs) directly.
- Support for the IADC standard DDR fields instead of free text only (Source 2).

17. Conclusion

Reading the Rig shows that an agentic workflow, rather than a single prompt, is what makes a small local model useful for this task. Splitting the job into planning, research, tool-based execution and self-checking, with a human approval step built into the graph, gave exact and traceable numbers on every synthetic report, caught every planted data error, and left the engineer with two clear, quick corrections instead of a page of free text to read. Every number is computed by Python, every flag is sourced to a report line, and every report is approved by a person.

References

[1] A Comparative Analysis of the Trouble Time for Conventional and Casing Drilling Wells: A Case Study on the Lobo Trend. Robert Gordon University OpenAIR repository. rgu-repository.worktribe.com. [Authors to be added by the team.]

[2] Daily Drilling Report Overhaul Aims to Provide Standard Framework for More Detailed Activity Tracking. Drilling Contractor (IADC), May/June 2020. drillingcontractor.org

[3] Differential Stuck Pipe Events: Development of Digital Best Practices for NPT Prevention. Montanuniversität Leoben. pureadmin.unileoben.ac.at. [Authors to be added by the team.]

[4] From Copilot to Co-Worker: Is Oil & Gas Ready to Trust Agentic AI to Execute Operational Decisions? SPE ATCE 2026 Technical Program. atce.org

[5] LangGraph Documentation. LangChain. docs.langchain.com

[6] Ollama: run large language models locally. ollama.com

[7] Streamlit Documentation. docs.streamlit.io

[8] LangChain Documentation. docs.langchain.com

[9] G. Aerts, A. Brun and M. Jerkø. How to Achieve 50 Percent Reduction in Offshore Drilling Costs. McKinsey & Company, May 2015. mckinsey.com

[10] J. Pallanich. 2026 Offshore Challenge: Softening Demand Puts the Brakes on Day Rates. Journal of Petroleum Technology (SPE), November 2025. jpt.spe.org

[11] AIQ Announces $340 Million Contract From ADNOC for Deployment of Agentic AI. Journal of Petroleum Technology (SPE), May 2025. jpt.spe.org

[12] Qwen Team. Qwen2.5 model family. qwenlm.github.io

Appendix A. How to run

Online, without installing anything: https://rig-sight.streamlit.app/ (mock classifier). Locally:

```
pip install -r requirements-dev.txt
python data/generate_data.py        # optional: rebuild the dataset
python -m pytest                    # automated tests
streamlit run app.py                # the PoC
ollama pull qwen2.5:3b              # optional local model
python evaluate.py                  # writes results/metrics_<backend>.json
```